

0.67. Les variables de la lettre dans leur propre fichier

Le 0.66, rangé en deux : `x`, `y` et `attente` déménagent dans « `variables.h` » ; `principal.cpp` n'a plus que la boucle du jeu.

1. Ce qu'il faut comprendre

C'est exactement le programme du 0.66, avec un seul changement : les **trois variables** du haut (`x`, `y`, `attente`) ne sont plus dans `principal.cpp`. Elles sont dans un **second fichier**, `variables.h` : c'est l'**onglet** à côté de `principal.cpp`, au-dessus du code.

Ce qui est nouveau ici : un programme en deux fichiers. `principal.cpp` est le fichier **principal** : c'est par lui que la compilation commence. `variables.h` est un fichier **voisin** : il ne fait rien tout seul, il attend qu'on le verse quelque part.

`#include "variables.h"` veut dire : « **verse ici** tout ce qu'il y a dans `variables.h` ». Avant de compiler, la console remplace cette ligne par le contenu du fichier, mot pour mot. Le programme compilé est donc **exactement le même** qu'au 0.66 : il est seulement rangé en deux.

Pourquoi tout en haut : une variable doit être déclarée **avant** qu'on s'en serve. `main()` utilise `x`, `y` et `attente` : le `#include` qui les apporte vient donc avant `main()`.

Le `.h` est une habitude du C : il veut dire « **header** », en-tête, un fichier fait pour être versé en haut d'un autre. Les guillemets autour du nom disent que le fichier est **à côté** du programme.

À quoi ça sert : d'un côté **l'état du jeu** (où est la lettre, combien attendre), de l'autre **ce qu'on en fait** (la boucle, les boutons). Pour changer la place de départ de la lettre, on n'ouvre que `variables.h`, sans toucher à la boucle.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **393 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
démarrage      60 o ← la console, à l'allumage — la mise en route de la console
main()         149 o ← démarrage — ta fonction
VBlank          8 o ← la console, à chaque image — compte les images, 60 fois par seconde
```

```

AttendreImage    28 o ← image() – attend l'image suivante
AttendreVBlank   16 o ← démarrage – attend que l'écran soit libre
AttendreAcces    12 o ← EffacerCases – attend de pouvoir écrire une case
AdresseCase      10 o ← effacer(), poser() – trouve l'adresse d'une case de l'écran
EffacerCases     12 o ← effacer() – efface des cases
CopierTuiles     31 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte     14 o ← démarrage – vide les deux cartes de l'écran
LireManette      37 o ← bouton() – lit les huit boutons
Tuiles           16 o ← CopierTuiles – 1 lettre sur 43

```

Lettres gravées : A

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

```

bouton() × 4    20 o sur place, dans main() + LireManette (37 o)
effacer() × 1   15 o sur place, dans main() + AdresseCase (10 o), EffacerCases (12 o)
poser() × 1     13 o sur place, dans main() + AdresseCase (10 o)
image() × 1      3 o sur place, dans main() + AttendreImage (28 o)

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// Le changement : les trois variables ne sont plus ici.
//
// #include "variables.h"
// |           |
// |           +-- le fichier à verser : l'onglet « variables.h », à côté
// +----- « verse ici » : avant la compilation, cette ligne est
//           remplacée par tout le contenu de variables.h. Tout se passe
//           comme si x, y et attente étaient écrits ici, comme au 0.66.

#include <effacer>    // efface des cases, ou tout le fond
#include <bouton>     // lit un bouton de la manette
#include <poser>      // pose une tuile sur une case du fond
#include <ALPHABET>   // les lettres de la police : ALPHABET[0] est le A
#include "variables.h"

```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```

int main() {
    // La boucle du jeu : exactement celle du 0.66. Elle se sert de x, y et
    // attente, qui viennent de variables.h.
    while (true) {
        image();                                // attend l'image suivante

        if (attente > 0) attente--;              // le temps passe

        if (attente == 0) {                     // on peut faire un pas
            effacer(x, y, 1);                    // efface l'ancienne place
            if (bouton(DROITE) && x < 19) { x++; attente = 8; }
            if (bouton(GAUCHE) && x > 0) { x--; attente = 8; }
            if (bouton(BAS) && y < 17) { y++; attente = 8; }
            if (bouton(HAUT) && y > 0) { y--; attente = 8; }
        }

        poser(x, y, ALPHABET[0]);                // la lettre à sa place
    }
}

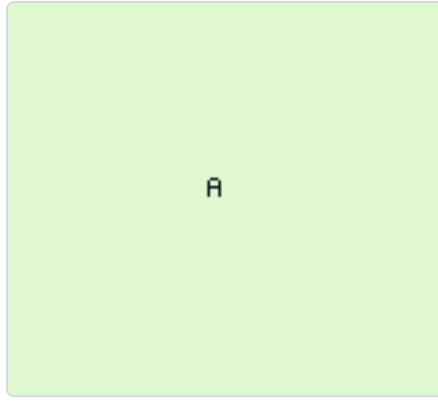
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



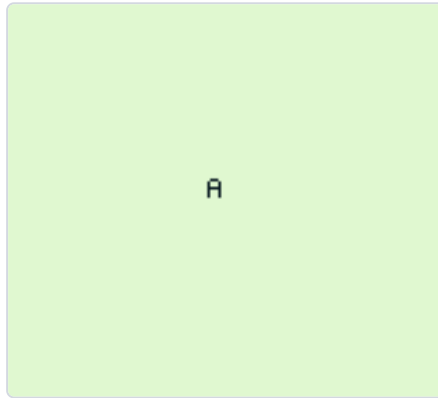
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

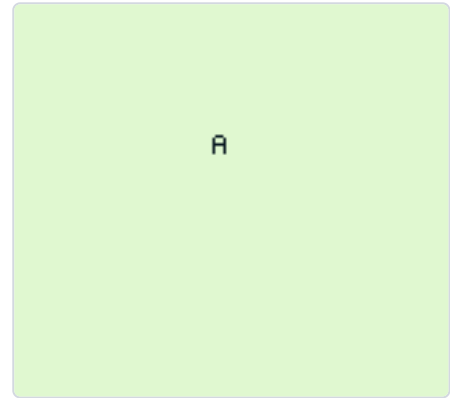
Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



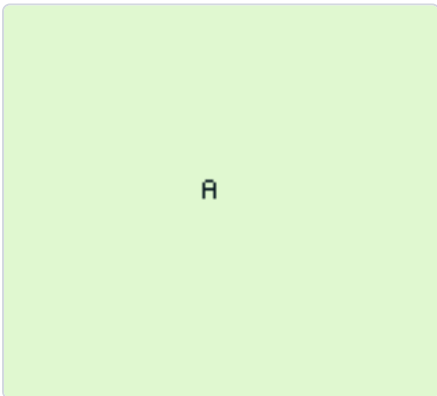
DROITE — l'écran a changé



GAUCHE — l'écran a changé



HAUT — l'écran a changé



BAS — l'écran a changé

5. Ce qu'on doit voir

Exactement le 0.66 : un A au milieu de l'écran, qui se déplace avec les flèches. Mais le programme est rangé en deux onglets.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ x et y viennent de `variables.h` : le A part du milieu (9, 8)

✓ la flèche droite le déplace

✓ la flèche bas aussi

✓ il est à sa nouvelle place

Ensuite : **0.67.1. Les variables de la lettre dans leur propre fichier — de base, ailleurs** — chapitre 0 : Avant tout. · [gameboy3](#) — écrire une cartouche Game Boy en C++